

Fair Drop: Product Requirements Document

Problem statement: Bit N Build, Web/App Dev PS 3: "Fair Drop: Selling 500 Seats to 50,000 People Without Letting Bots Win" **Team:** Naytik Shah (Backend), Jash Madhani (Frontend), Zeal Shah (Bot defence and adversarial testing) **Version:** v1.0, 3 Oct 2026 **Status:** Draft. The API section is v1 and will be kept in sync with the code as we build.

1. Problem

High-demand drops (tickets, registrations, limited seats) are usually first-come-first-served. That turns the sale into a race, and races are won by whoever has the fastest connection or the most automated clients. Bots send thousands of requests at the opening second, the servers buckle, and real people get timeouts, broken carts and inconsistent results.

As long as arrival time decides who wins, bots will always have an edge. So our approach is to remove speed from the equation entirely.

2. Core idea

Fair Drop replaces the race with a **registration window followed by a weighted, verifiable lottery**.

1. **Speed does not matter.** Anyone who enters during the window has the same base chance, whether they entered in the first second or the last hour.
2. **The only way to cheat a lottery is to hold more tickets.** So the defence focuses on making each extra fake identity expensive (one entry per payment method) and on finding fake-identity clusters before the draw.
3. **Suspicious entries are down-weighted, not banned.** A real user who gets wrongly flagged keeps some chance, while a bot farm gets diluted to almost nothing.
4. **The draw can be checked.** The random seed is committed before entries open and revealed after the draw, so anyone can re-run it and get the same result.
5. **Tickets are named and non-transferable.** This removes most of the resale incentive that motivates bot operators in the first place.

We do not claim to stop every bot. The goal, as stated in the PS, is that bots gain no significant advantage, and we measure exactly how much advantage they get.

3. Goals and non-goals

Goals

- Bots gain no meaningful advantage through speed, request volume or repeated attempts.
- Zero overselling, zero duplicate entries and zero duplicate allocations, under any load.

- The system stays up and responsive during a simulated flash crowd of 50,000 users.
- User state survives refreshes, reconnects and temporary failures.
- Fairness is measurable: configurable attack scenarios produce clear numbers.

Non-goals (for this hackathon)

- Real money movement. Payments go through a mock service (Razorpay test mode is a stretch goal).
- Real SMS. Email OTP is used in the demo; phone OTP is the intended production channel.
- Government ID or KYC checks.
- Seat maps or choosing specific seats.
- Resale or transfer marketplace.
- Native mobile apps.
- Multi-region deployment.

4. Users

User	What they need
Participant	Enter a drop easily, know their status at all times, trust that the result was fair.
Organiser	Configure a drop, watch it live, review flagged entries with evidence, run the draw, see fairness metrics.
Verifier (anyone, including judges)	Independently confirm the draw was not rigged.
Attacker (simulated)	Win as many seats as possible using automation, multiple accounts and retries.

5. Drop lifecycle

```
ANNOUNCED -> OPEN -> CLOSED -> SCORING -> DRAWN -> CONFIRMING ->
COMPLETE
(seed hash      (entries  (entries  (risk      (ranked  (winners
confirm,
published)      accepted) frozen,   scores,   order     waitlist
promoted
                manifest  weights) computed, on expiry)
                published) seed
                                revealed)
```

State	What happens
ANNOUNCED	Drop is created from config. <code>SHA-256(seed)</code> is published.
OPEN	Participants sign in and enter. Rate limits, proof-of-work and the payment hold apply.
CLOSED	No new entries. The entry list is frozen.
SCORING	Risk signals are computed and every entry gets a weight. The draw manifest (entry IDs + weights) is hashed and published.
DRAWN	The weighted draw runs with the committed seed. Every entry gets a rank. The seed is revealed.
CONFIRMING	Ranks 1 to N (N = seats) are winners and must confirm within the confirm window. Unconfirmed or failed winners are replaced by the next rank on the waitlist.
COMPLETE	All seats are confirmed or the waitlist is exhausted. Holds are released for everyone who did not get a seat.

6. User flows

6.1 Participant

1. Opens the drop page and sees seats, window times and the published seed hash.
2. Requests an OTP with their email. The browser solves a small proof-of-work first.
3. Verifies the OTP and gets a session (HTTP-only cookie).
4. Clicks Enter. The browser solves a proof-of-work challenge, collects a device fingerprint and the user adds a payment method (test values in the demo).

5. A refundable hold is placed and the entry is confirmed. One entry per account and one entry per payment method.
6. Waits. The status page updates live over Server-Sent Events (SSE). Refreshing or reconnecting shows the same status.
7. After the draw, sees one of: **Won (confirm by time X)**, **Waitlisted (position #k)** or **Not selected (hold released)**.
8. If won, confirms with the same payment method. The hold is captured and a named ticket is issued.

6.2 Organiser

1. Defines the drop in `config/drops.yaml` (seats, window, confirm window, hold amount, weights).
2. Watches the live dashboard: entries, request rates, rate-limit hits, PoW failures, errors.
3. Runs scoring after the window closes and reviews flagged entries with evidence.
4. Triggers the draw.
5. Watches confirmations and waitlist promotions, then reads the fairness metrics.

6.3 Verifier

1. Downloads the manifest and checks its hash against the one published at close.
2. Checks that the revealed seed hashes to the commitment published at announcement.
3. Re-runs the draw algorithm (open source, also runnable on the verify page in the browser) and compares the ranking.

7. Functional requirements

Mapped one-to-one to the key features in the problem statement.

7.1 High-Concurrency Support

- **FR-1.1** Entries are spread across a registration window, so there is no single-second spike for entry submission.
- **FR-1.2** The API is stateless apart from Redis and Postgres and runs as multiple replicas behind a load balancer.
- **FR-1.3** Postgres connections are pooled. Hot reads (drop details, published results, per-user status) are served from Redis.
- **FR-1.4** Results are computed once and cached. The reveal burst only does key lookups.
- **FR-1.5** Live updates go through SSE with Redis pub/sub for fan-out across replicas, so clients do not poll.

7.2 Abuse Handling

- **FR-2.1 Identity anchor:** each entry requires a refundable payment hold. One entry per payment method per drop, enforced by a unique constraint on a keyed hash of the payment method fingerprint.
- **FR-2.2 OTP:** email OTP in the demo (phone in production). One account per verified email. Disposable email domains are rejected.
- **FR-2.3 Proof-of-work:** OTP requests and entry submissions require a solved hashcash-style challenge. Difficulty is adaptive per IP and subnet, and is raised for sources under heavy load.
- **FR-2.4 Rate limiting:** Redis token buckets per IP, per subnet, per session and per email (defaults in Section 9.4).
- **FR-2.5 Idempotency:** every mutating request carries an `Idempotency-Key`. Retries return the original response instead of creating new records.
- **FR-2.6 Risk scoring:** after the window closes, each entry is scored on four signals: device fingerprint overlap, IP/subnet clustering, registration timing patterns, and email/name similarity (Section 9.2).
- **FR-2.7 Down-weighting:** risk scores map to draw weights. No entry is removed purely on its risk score.

7.3 Allocation Integrity

- **FR-3.1 Unique constraints:** `((drop_id, user_id))` and `((drop_id, anchor_hash))` on entries; `((drop_id, entry_id))` and `((drop_id, seat_no))` on allocations.
- **FR-3.2 Seat assignment and waitlist promotion** run inside a transaction that locks the drop row (`SELECT ... FOR UPDATE`) and checks that active allocations never exceed seats.
- **FR-3.3 Payment capture and allocation confirmation are linked:** a ticket is issued only after capture succeeds, and capture is retried idempotently.
- **FR-3.4 Every state change is written to an append-only audit log** where each row stores the hash of the previous row, so tampering breaks the chain.

7.4 Reliable Sessions

- **FR-4.1 Sessions live in Redis** behind an HTTP-only, Secure, SameSite cookie. Sessions can be revoked instantly.
- **FR-4.2 Postgres is the source of truth** for entry and allocation state. The client never holds state that the server does not.
- **FR-4.3 SSE events carry IDs.** On reconnect, the browser sends `Last-Event-ID` and missed events are replayed from a short Redis stream.
- **FR-4.4 If SSE drops completely,** the client falls back to `GET /entries/me` with backoff.

- **FR-4.5** Winners have a 10 minute confirm window. This keeps the waitlist moving quickly, and SSE reconnect plus the status fallback (FR-4.3, FR-4.4) mean a brief disconnect does not hide a win from the user.

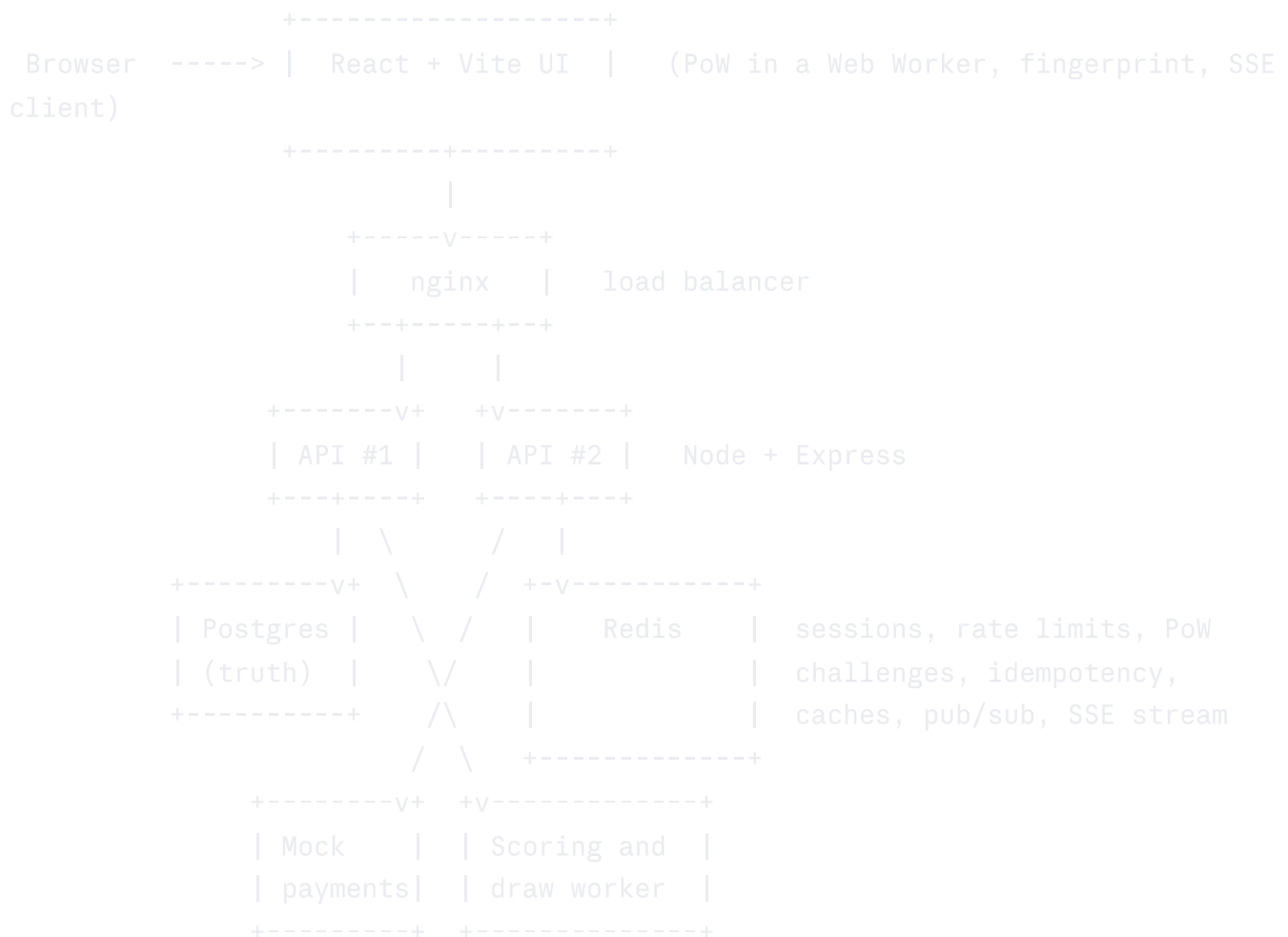
7.5 Adversarial Testing

- **FR-5.1** A custom Node bot simulator runs configurable honest and attacker populations against the real API.
- **FR-5.2** k6 runs raw flash-crowd load (window open, results reveal).
- **FR-5.3** Every defence layer can be toggled per run, so each layer's contribution can be measured.

7.6 Fairness Measurement

- **FR-6.1** Each simulator run writes a metrics JSON file. The organiser dashboard reads and charts it.
- **FR-6.2** Metrics and pass/fail targets are defined in Section 11.

8. Architecture



Component	Tech	Responsibility
Web app	React + Vite	Participant pages, organiser dashboard, draw verify page
API	Node.js + Express	Auth, entries, SSE, admin endpoints
Database	PostgreSQL	Users, drops, entries, allocations, flags, audit log
Cache and coordination	Redis	Sessions, rate limits, PoW challenges, idempotency keys, caches, pub/sub
Mock payments	Node service	Payment methods, holds, capture, release, failure injection
Worker	Node	Risk scoring, manifest, draw, waitlist expiry
Load balancer	nginx	Spreads traffic across API replicas
Testing	Custom Node simulator + k6	Adversarial scenarios and load

Deployment: Docker Compose runs the whole stack locally for load and attack tests (free cloud tiers throttle and would distort results). A free hosted instance of the UI and API is used for the live demo link.

9. Key designs

9.1 Weighted, verifiable draw

- At announcement, the server generates a 256-bit random `(seed)` and publishes `(seedCommit = SHA-256(seed))`.
- At close, the manifest is built: every entry as `{ entryId, weight }`, sorted by `(entryId)`. Entry IDs are random UUIDs and reveal nothing about users. `(manifestHash = SHA-256(canonical JSON of manifest))` is published before the draw.
- For each entry: `(u = (first 52 bits of HMAC-SHA256(seed, entryId) + 1) / (2^52 + 1))`, then `(key = ln(u) / weight)`.
- Entries are sorted by `(key)` descending (ties broken by `(entryId)`). Rank 1 to N are winners; the rest form the waitlist in order.
- This is the Efraimidis-Spirakis method for weighted sampling without replacement. It gives a full ranking from one seed, so the waitlist is as verifiable as the winner list.

- After the draw, `seed` is revealed. Anyone can check `SHA-256(seed) == seedCommit`, check the manifest hash, and re-run the ranking.

Known limitation: the organiser knows the seed while weights are assigned. The stretch goal replaces the self-generated seed with a public external value that only becomes known after the window closes.

9.2 Risk scoring

Each signal produces a score from 0 to 1.

Signal	What we look at
Device fingerprint overlap	How many entries share a fingerprint. 1 entry scores 0, 2-3 entries score medium, 4 or more score high.
IP / subnet clustering	Entries per /24 (IPv4) or /64 (IPv6) compared with the median. Given a lower signal weight because colleges and offices share networks.
Registration timing	Regular gaps between entries within a cluster, tight bursts, unusually fast OTP-to-entry times, and proof-of-work solved faster than normal hardware allows.
Email / name similarity	Normalised emails (dots and plus tags removed), sequential numbering, small edit distances between names, disposable domains.

$\text{risk} = 100 \times (0.35 \times \text{device} + 0.25 \times \text{ip} + 0.20 \times \text{timing} + 0.20 \times \text{email})$, plus a cluster bonus when an entry belongs to a group of linked entries (sharing two or more signals) above a size threshold. All coefficients live in config and are calibrated on simulator data.

Risk score	Weight	Label
0-29	1.00	Clean
30-59	0.50	Low
60-84	0.20	Medium
85-100	0.05	High

Every flag stores its evidence: which signals fired, their values, and the IDs of linked entries. The dashboard shows this so each decision can be explained.

9.3 Proof-of-work

- The client gets `{ challengeId, prefix, difficulty }` and finds a `nonce` such that `SHA-256(prefix + nonce)` starts with `difficulty` zero bits.
- Solved in a Web Worker so the page stays responsive.
- Challenges are single-use, expire after 2 minutes and are stored in Redis.
- Base difficulty is configurable (starting at 18 bits) and tuned so a mid-range phone solves it in about 1-2 seconds. Difficulty rises for an IP or subnet as its request rate grows, with a cap so real users on shared networks are not locked out.

9.4 Rate limits (defaults, configurable)

Scope	Endpoint group	Limit
IP	All	60 req/min
/24 subnet	All	600 req/min
IP	OTP request	5 per 10 min
Email	OTP request	3 per 10 min
Session	Entry submit	5 per min
Session	Status reads	120 per min

Responses over the limit return `429` with a `Retry-After` header.

9.5 Mock payment service

- Supports creating a payment method from test values, then hold, capture and release.
- Returns a stable `fingerprint` per payment method (same card or UPI ID gives the same fingerprint). The API stores only `HMAC(server_pepper, fingerprint)`.
- No real card or UPI data is ever used or stored.
- Failure injection (declines, timeouts, slow responses) is configurable for testing.
- The interface matches what a real gateway needs, so Razorpay test mode can replace it later.

10. API specification (v1)

Base path: `/api/v1`. JSON in and out. Auth is the session cookie unless marked public. All mutating requests require the header `Idempotency-Key: <uuid>`.

Error format

json

```
{ "error": { "code": "ANCHOR_ALREADY_USED", "message": "This payment method has al
```

Common error codes: `RATE_LIMITED` (429), `POW_INVALID` (400), `POW_EXPIRED` (400),
`UNAUTHENTICATED` (401), `FORBIDDEN` (403), `NOT_FOUND` (404), `DROP_NOT_OPEN` (409),
`ALREADY_ENTERED` (409), `ANCHOR_ALREADY_USED` (409), `PAYMENT_FAILED` (402),
`CONFIRM_WINDOW_EXPIRED` (410).

10.1 Auth

`GET /pow/challenge?purpose=otp|entry` (public)

json

```
{ "challengeId": "c_8f2a", "prefix": "a91f...", "difficulty": 18, "expiresAt": "20
```

`POST /auth/otp/request` (public)

json

```
{ "email": "user@example.com", "pow": { "challengeId": "c_8f2a", "nonce": "193847"
```

Response `202`. The same response is returned whether or not the email exists.

`POST /auth/otp/verify` (public)

json

```
{ "email": "user@example.com", "code": "482913" }
```

Response `200` with `Set-Cookie: sid=...; HttpOnly; Secure; SameSite=Lax`.

json

```
{ "user": { "id": "u_12", "email": "user@example.com", "role": "participant" } }
```

`POST /auth/logout` Response `204`.

`GET /me`

json

```
{ "id": "u_12", "email": "user@example.com", "role": "participant" }
```

10.2 Drops

GET /drops (public)

json

```
{ "drops": [ { "id": "d_1", "name": "Launch Night", "seats": 500, "state": "OPEN",  
              "windowOpensAt": "...", "windowClosesAt": "..." } ] }
```

GET /drops/:dropId (public)

json

```
{ "id": "d_1", "name": "Launch Night", "seats": 500, "state": "OPEN",  
  "windowOpensAt": "...", "windowClosesAt": "...", "confirmWindowMinutes": 10,  
  "holdAmount": 100, "currency": "INR", "seedCommit": "3b7e..." }
```

10.3 Entries

POST /drops/:dropId/entries

json

```
{ "pow": { "challengeId": "c_91bd", "nonce": "77120" },  
  "paymentMethodToken": "pm_test_4242",  
  "deviceFingerprint": "fp_a8c3..." }
```

Response **(201)**:

json

```
{ "entryId": "e_5c1f", "state": "ENTERED", "holdId": "h_77" }
```

Errors: **DROP_NOT_OPEN**, **ALREADY_ENTERED**, **ANCHOR_ALREADY_USED**, **PAYMENT_FAILED**,
POW_INVALID.

GET /drops/:dropId/entries/me

json

```
{ "entryId": "e_5c1f",  
  "state": "ENTERED | WON | WAITLISTED | NOT_SELECTED | CONFIRMED | EXPIRED",  
  "rank": 42, "waitlistPosition": null, "confirmBy": "2026-10-03T18:30:00Z" }
```

POST /drops/:dropId/entries/me/confirm

json

```
{ "paymentMethodToken": "pm_test_4242" }
```

The payment method must match the original anchor. Response **200**:

json

```
{ "ticket": { "id": "t_301", "dropId": "d_1", "holderName": "As on account", "seatNo": 301 } }
```

Errors: **CONFIRM_WINDOW_EXPIRED**, **PAYMENT_FAILED**, **FORBIDDEN** (anchor mismatch).

GET /tickets/me

json

```
{ "tickets": [ { "id": "t_301", "dropId": "d_1", "seatNo": 301 } ] }
```

10.4 Live updates (SSE)

GET /drops/:dropId/events with **Accept: text/event-stream**. Supports **Last-Event-ID**.

Event	Payload
drop_state	{ "state": "CLOSED" }
manifest_published	{ "manifestHash": "..." }
draw_complete	{ "seed": "...", "resultsReady": true }
your_status	{ "state": "WON", "rank": 42, "confirmBy": "..." }
waitlist_moved	{ "waitlistPosition": 7 }
heartbeat	{ } every 20 s

10.5 Draw verification (public)

GET /drops/:dropId/draw

json

```
{ "seedCommit": "3b7e...", "seed": "9a0c... (null until revealed)",
  "manifestHash": "e41d...", "algorithm": "es-weighted-v1",
  "manifestUrl": "/api/v1/drops/d_1/draw/manifest" }
```

GET /drops/:dropId/draw/manifest

json

```
{ "entries": [ { "entryId": "e_0003", "weight": 1.0 }, { "entryId": "e_0007", "wei
```

GET /drops/:dropId/draw/results

json

```
{ "ranking": [ "e_5c1f", "e_0912", "..." ], "seats": 500 }
```

10.6 Organiser (role: organiser)

GET /admin/drops/:dropId/live

json

```
{ "entries": 41233, "entriesPerMin": 812, "rateLimited": 15420, "powFailures": 331,
  "errors5xx": 4, "p95LatencyMs": 182, "activeSseClients": 38004 }
```

POST /admin/drops/:dropId/score Runs risk scoring. Response (202) with { "jobId": "j_4" }.

GET /admin/drops/:dropId/flags?minScore=60&page=1

json

```
{ "flags": [ { "entryId": "e_88a1", "risk": 91, "weight": 0.05, "clusterId": "cl_3",
  "page": 1, "total": 2310 }
```

GET /admin/drops/:dropId/flags/:entryId

json

```
{ "entryId": "e_88a1", "risk": 91,
  "signals": { "device": 1.0, "ip": 0.7, "timing": 0.9, "email": 0.8 },
  "evidence": { "sharedFingerprintWith": 139, "subnet": "203.0.113.0/24",
    "medianGapMs": 410, "emailPattern": "seq-number" },
  "linkedEntries": [ "e_88a2", "e_88a3" ] }
```

POST /admin/drops/:dropId/draw Freezes the manifest if not already frozen, runs the draw with the committed seed and reveals it. Response (200) with { "manifestHash": "...", "seed": "..." }.

GET /admin/drops/:dropId/metrics?runId=... Returns the metrics from Section 11 for a simulator run.

GET /admin/audit?dropId=d_1&cursor=...

json

```
{ "events": [ { "seq": 9921, "type": "WAITLIST_PROMOTED", "entryId": "e_0912",
  "at": "...", "prevHash": "...", "hash": "..." } ], "nextCursor": "
```

10.7 Mock payment service (internal)

Method	Path	Purpose
POST	<code>/payments/methods</code>	Create a test payment method. Returns <code>{ token, fingerprint }</code> .
POST	<code>/payments/holds</code>	<code>{ token, amount, reference }</code> returns <code>{ holdId, status: "HELD" }</code>
POST	<code>/payments/holds/:holdId/capture</code>	Capture the hold
POST	<code>/payments/holds/:holdId/release</code>	Release the hold
PUT	<code>/payments/_config</code>	Failure injection settings (test only)

11. Evaluation

11.1 Scenarios

Run with the default scale (500 seats, 50,000 users) unless stated. Scale is configurable.

Scenario	Attacker behaviour
S0 Baseline	Honest users only
S1 Naive farm	Many accounts from one device and subnet, fixed timing, sequential emails
S2 Stealth farm	Rotated IPs, randomised timing, varied emails, a smaller pool of payment methods
S3 Retry spammer	Repeats entry and OTP requests aggressively
S4 Flooder	High request volume, skips or fakes proof-of-work
S5 Anchor reuse	Tries to enter many times with the same payment method
S6 Mixed	All of the above, with bots at 10%, 30% and 50% of entries

Each scenario runs with defences on, and again with individual layers turned off, to show what each layer contributes.

11.2 Metrics and targets

Hard guarantees (must hold in every run):

Metric	Target
Oversold seats	0
Users holding more than one seat	0
Entries per payment method per drop	at most 1
Draw reproducibility from published seed and manifest	100%
Reconnecting users shown the correct status	100%

Goals:

Metric	Definition	Target
Bot advantage ratio (S1)	bot share of winners / bot share of entries	at most 0.2
Bot advantage ratio (S2)	same	at most 0.5
Honest fair-share deviation	honest win rate vs seats / honest entries	within 10%
False positives	honest entries in Medium or High tiers	at most 5%
Entry submit latency	p95 at 50k users across the window	under 500 ms
Results lookup latency	p95 during the reveal burst	under 200 ms
Server error rate	5xx responses (429s excluded)	under 0.5%

Results are reported with the hardware they were measured on, since local runs depend on the machine.

12. Data model

Table	Key columns
users	id, email (unique), name, role, created_at
drops	id, name, seats, state, window_opens_at, window_closes_at, confirm_window_min, hold_amount, seed_commit, seed, manifest_hash
entries	id, drop_id, user_id, anchor_hash, hold_id, device_fp, ip, subnet, pow_solve_ms, created_at, state, risk, weight, rank. Unique (drop_id, user_id), unique (drop_id, anchor_hash)
risk_signals	entry_id, device, ip, timing, email, cluster_id, evidence (jsonb)
allocations	id, drop_id, entry_id, seat_no, state, confirm_by, ticket_id. Unique (drop_id, entry_id), unique (drop_id, seat_no)
tickets	id, allocation_id, holder_user_id, issued_at
audit_log	seq, drop_id, type, payload (jsonb), at, prev_hash, hash

Redis keys: `sess:<sid>`, `rl:<scope>:<key>`, `pow:<challengeId>`, `idem:<key>`, `drop:<id>:summary`, `drop:<id>:status:<userId>`, `drop:<id>:events` (stream).

13. Build plan

Clock starts at H0. Prototype due at **H1**, final submission at **H12**. Feature freeze at **H10:30**.

Time	Naytik (Backend)	Jash (Frontend)	Zeal (Bot defence and testing)
H0-H1 Prototype	Express skeleton, Postgres schema with unique constraints, simple OTP, entry endpoint, plain random draw, results endpoint	Vite app: sign in, drop page, enter button, status page (against agreed API contracts)	Simulator skeleton (honest users + naive farm), basic k6 script, signal definitions
H1-H3	Redis sessions, rate limits, PoW, idempotency keys, mock payment service with holds	Email OTP flow, PoW solver in Web Worker, fingerprint collection, payment step	Farm strategies, signal extraction queries
H3-H5	Risk scoring job, weights, commit-reveal, weighted draw, manifest	SSE status updates, results page, reconnect handling	Calibrate scoring on simulator data, check false positives
H5-H7	Winner confirm, waitlist promotion, capture and release, hash-chained audit log	Organiser dashboard: live view, flags with evidence	Stealth farm, retry spammer, flooder, metrics JSON output
H7-H9	nginx with multiple replicas, connection pooling, result caching, admin endpoints	Metrics charts, draw verify page (re-run in browser)	Full k6 run at 50k, collect numbers
H9-H10:30	Bug fixes from test runs, hosted deploy	Polish, hosted deploy	All scenarios S0 to S6, final metrics
H10:30-H12	README, architecture notes	Demo video	Evaluation write-up

Dependencies: API contracts (Section 10) are frozen at H0:30 so frontend and simulator work can run in parallel. Any later change is announced in the team channel and updated here.

14. Risks

Risk	Mitigation
Organiser knows the seed during weighting	Manifest hash published before the draw; external public seed as stretch goal
Shared networks (college, office) look like bot clusters	IP gets a lower signal weight; signals are combined; down-weight instead of ban
Local hardware cannot reach 50k	Scale is configurable; results reported with hardware details
PoW is slow on low-end phones	Web Worker, capped difficulty, difficulty only raised for heavy sources
Well-funded attackers with many real cards	Raises cost per entry rather than blocking outright; named tickets remove resale value; measured in S2
Time pressure	Prototype at H1 covers the full happy path; later hours add layers on top of a working base

15. Stretch goals

- Razorpay test mode instead of the mock payment service.
- Public external seed for the draw.
- Phone OTP.
- Organiser override of individual flags with a logged reason.

16. Glossary

Term	Meaning
Identity anchor	Something costly to duplicate (here, a payment method) that limits one person to one entry
Proof-of-work	A small puzzle the browser solves before a request, making mass requests expensive
Commit-reveal	Publishing a hash of a secret first and the secret later, so it cannot be changed in between
Manifest	The frozen list of entry IDs and weights that the draw runs on
Bot advantage ratio	How much bots' share of winners exceeds or falls short of their share of entries
SSE	Server-Sent Events, a one-way live connection from server to browser